

Beyond Sequential Interaction: Benchmarking Parallel Execution and Coordination for GUI Agents

Zedong Yu*, Qianxing Li*, Zhi Gao†, Liuyu Xiang, Chenrui Shi, Yang Liu, Huiming Wu, Yujie Wei, Yuhao Fei, Yubo Fu, and Zhaofeng He†

Abstract—Graphical user interface (GUI) agents are systems powered by large multimodal models (LMMs). They perceive screen state and execute user instructions through GUI actions such as clicking, typing, and scrolling on desktops and mobile devices. However, current agents scale poorly to long-horizon tasks: actions incur costly LMM inferences, and performance degrades as context grows. Humans divide such workloads among collaborators who complete sub-tasks in parallel. Yet parallel coordination among GUI agents has received little attention. To close this gap, we introduce ParaGUIBench, to our knowledge, the first benchmark dedicated to parallel execution and coordination of multiple GUI agents on separate desktop instances. It consists of three components: a multi-device Docker infrastructure with a shared file system; a dataset of 233 tasks spanning six task categories; and an evaluation system with efficiency metrics, including step reduction ratio and token cost. We further introduce ParaGUI, a planner-worker agent that decomposes GUI tasks and dispatches sub-tasks to concurrent workers on separate desktop instances. On ParaGUIBench, ParaGUI reaches a 46.4% success rate, outperforming the strongest serial baseline (Claude Sonnet 4.6) by 12.9 points while using roughly half the steps and less than half the tokens. These results show that parallel execution can improve both success rate and efficiency on decomposable, long-horizon GUI tasks, pointing to a direction worth further study. Project page: <https://github.com/pkgunboat/ParaGUIBench>.

Index Terms—GUI agent, large multimodal model, parallel execution, multi-agent coordination, benchmark.

I. INTRODUCTION

GRAPHICAL user interface (GUI) agents use Large Multimodal Models (LMMs) to observe the screen and follow user instructions by clicking, typing, and scrolling on computers or phones [1]–[5]. They are part of a broader class of LMM-driven sequential decision systems, where perception, planning, and action are coupled in an interaction loop [6], [7]. Most GUI agents execute long-horizon tasks as a serial

chain of perception–decision–action cycles, with each cycle incurring a costly LMM inference. Abhyankar et al. [8] report that LMM inference accounts for 75–94% of per-task latency in GUI agents. Growing context length also makes later steps less accurate and up to $3\times$ slower than earlier ones. Model- and system-level acceleration, e.g. via quantization, pruning, and distillation [9], lowers per-call latency but leaves this serial loop structure untouched. Long trajectories thus limit the performance and efficiency of GUI agents.

Humans handle such tasks differently: they divide the workload among teammates, who complete independent sub-tasks in parallel on separate machines before combining their results. Yet parallel coordination among GUI agents on separate desktop instances remains largely unexplored. Standard benchmarks such as OSWorld [10] and WebArena [11] fix evaluation to a single-agent and single-device protocol. These single-agent, single-device benchmarks provide no infrastructure for concurrent execution, no tasks that require cross-worker coordination, and no metrics for parallel efficiency. This raises a basic question: *can GUI agents coordinate in parallel to accelerate long-horizon GUI workflows, and how do we measure such coordination?*

Parallel GUI execution introduces three challenges beyond merely dispatching multiple actions at once. First, the user instruction does not specify the dependencies between steps, so the system must identify which sub-tasks can run concurrently. Second, concurrent workers may conflict when they modify shared artifacts, such as the same file or overlapping output regions. Third, each worker sees only its own sub-task, so local completion does not guarantee that the combined result satisfies the original instruction.

In this paper, we propose ParaGUIBench, to our knowledge, the first benchmark dedicated to parallel execution and coordination of multiple GUI agents on separate desktop instances. It consists of a parallel-native multi-device Docker infrastructure, a curated dataset, and an evaluation system. The infrastructure runs concurrent workers that share a file system. The dataset contains 233 tasks across six categories grouped under two domains, information retrieval and operation & manipulation. The evaluation system reports task success rates and three efficiency metrics: step reduction ratio, parallelism degree, and token cost.

We further introduce ParaGUI, a planner-worker agent for parallel GUI automation, as shown in Figure 1. The planner decomposes a long-horizon GUI task into parallelizable sub-tasks. It then dispatches them to multiple GUI workers running concurrently on separate desktop instances, aggregates their

*Zedong Yu and Qianxing Li contributed equally to this work.

Zedong Yu, Liuyu Xiang, Yujie Wei, Yuhao Fei, Yubo Fu, and Zhaofeng He are with Beijing University of Posts and Telecommunications, Beijing, China (e-mail: yzd0812z2@bupt.edu.cn; xiangly@bupt.edu.cn; yujie.wei@bupt.edu.cn; 2022213322@bupt.cn; fuyubo23@bupt.edu.cn; zhaofenghe@bupt.edu.cn).

Zedong Yu, Qianxing Li, Zhi Gao, Chenrui Shi, Yang Liu, and Huiming Wu are with the State Key Laboratory for General Artificial Intelligence, BIGAI, Beijing, China (e-mail: liuyang1@bigai.ai; hwubl@connect.ust.hk).

Qianxing Li is with China University of Geosciences (Beijing), Beijing, China (e-mail: 1004236211@email.cugb.edu.cn).

Zhi Gao and Chenrui Shi are with Beijing Institute of Technology, Beijing, China (e-mail: gaozhibit@bit.edu.cn; shichenrui@bit.edu.cn).

†Corresponding authors: Zhi Gao and Zhaofeng He (e-mail: gaozhibit@bit.edu.cn; zhaofenghe@bupt.edu.cn).

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

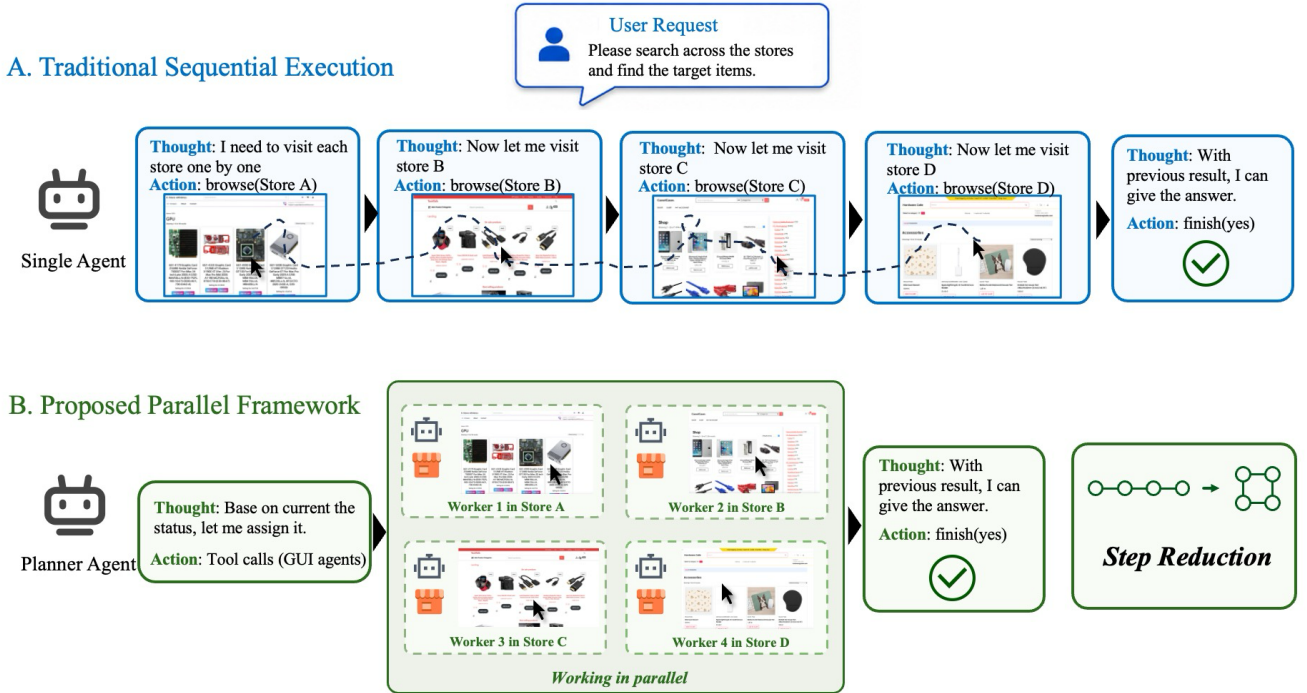


Fig. 1. Sequential vs. parallel GUI execution. (A) A traditional agent runs a long-horizon task as one serial chain of perception–decision–action steps. (B) ParaGUI’s planner decomposes the task and dispatches sub-tasks to parallel GUI workers (four shown; default $N=5$), each in its own isolated Docker container, then aggregates their results—shortening the critical path whenever the task can be split.

returned summaries, and decides whether to dispatch another round or terminate. ParaGUI delivers a +12.9 percentage-point success-rate gain with roughly half the steps and less than half the tokens of the strongest serial baseline. In addition, on standard serial OSWorld, ParaGUI outperforms a strong GUI-only baseline, indicating that its planner–worker design transfers beyond parallel-native tasks. Using ParaGUI to analyze ParaGUIBench, we find that exploitable parallelism is governed primarily by task dependency structure rather than application domain, while errors within individual GUI workers form the largest failure category.

Contributions. (1) We present ParaGUIBench, to our knowledge the first benchmark dedicated to parallel execution and coordination of multiple GUI agents on separate desktop instances, providing the infrastructure, tasks, and metrics. (2) We develop ParaGUI, a planner–worker agent for parallel GUI execution that combines adaptive round-based planning with self-contained dispatch—each dispatched instruction contains every earlier result its sub-task depends on, so the worker needs nothing else to run. (3) We systematically characterize when parallel GUI execution is beneficial, how system components shape capability and efficiency, and why failures occur.

II. RELATED WORK

a) GUI agent benchmarks and efficiency-aware evaluation: LMM-powered GUI agents have driven a wave of benchmarks. WebArena [11] and VisualWebArena [12] target web tasks; Mind2Web [13] targets offline generalization; OSWorld [10] and WindowsAgentArena [14] target live desktop;

AndroidWorld [15] targets mobile. All of them run a single agent on a single device, execute one step at a time, and report the success rate as the main metric. None evaluates coordination, and efficiency is rarely a primary metric. A few works look at cost: Abhyankar et al. find current agents are 10–20× slower than humans [8], and HAL [16] and WABER [17] add cost and latency to their metrics. These benchmarks that focus on efficiency still assume a single sequential agent rather than multiple agents in parallel. In contrast, ParaGUIBench evaluates parallel execution and coordination across multiple GUI agents, complementing success rate with step-reduction and parallelism metrics that these single-agent benchmarks omit.

b) Parallel and multi-agent systems: Prior work explores parallel execution for LLM agents at the API level (such as LLMCompiler [18] and DynTaskMAS [19]) and at the planning level (such as Plan-over-Graph [20] and Flow [21]). Frameworks such as MetaGPT [22] and MegaAgent [23] organize agents into role-based workflows. A separate multi-agent reinforcement learning literature studies coordination through communication and task allocation [24], [25], but mainly in learned-control settings rather than benchmarked GUI workflows. The closest adjacent efforts are UFO³ [26] and Flash-Searcher [27]. UFO³ [26] evaluates asynchronous coordination across heterogeneous Windows and Linux endpoints, where parallelism is organized primarily across device agents and its Windows endpoint itself combines GUI and application-API control. Flash-Searcher [27] parallelizes web

TABLE I
COMPARISON OF REPRESENTATIVE SYSTEMS AND BENCHMARKS FOR PARALLEL AGENT EXECUTION.

Work	Primary target	Parallel unit	Interaction interface	Evaluation emphasis
OSWorld [10]	Single-agent GUI control	None (single serial agent)	Screen, mouse, keyboard	Task success
MultiAgentBench [31]	Multi-agent coordination	LLM agents	Text / API calls	Success, coordination
Collab-Overcooked [32]	Cooperative planning	Agents in a gridworld	Symbolic actions	Success, coordination
Flash-Searcher [27]	Parallel web retrieval	DAG reasoning / tool branches	Search and crawl APIs	Accuracy, execution steps
UFO ³ / NebulaBench [26]	Cross-device coordination	Heterogeneous device agents	CLI + hybrid GUI/API	Success, parallelism, latency, robustness
ParaGUI / ParaGUIBench (ours)	Parallel GUI execution	Multiple GUI workers	Screen, mouse, keyboard	Success, critical-path steps, parallelism, tokens

information retrieval through search and crawl APIs and reports execution-step efficiency, but involves no screen-grounded interaction or GUI-state modification. ParaGUIBench instead directly evaluates multiple GUI agents operating concurrently on separate desktop instances, with a rule-based evaluator that checks each task outcome against textual answers or the resulting environment state. On the evaluation side, PLaG [28], Robotouille [29], AsyncTool [30], MultiAgentBench [31], and Collab-Overcooked [32] test asynchronous planning and coordination in *non-GUI, abstracted environments*—text-based games, symbolic task graphs, and gridworld-style kitchens—where actions are high-level API calls rather than pixel-level GUI operations. Different from them, ParaGUIBench grounds parallel coordination in *real, stateful GUI environments*, where agents act through screen-grounded controls and may contend over persistent shared artifacts. Table I contrasts these systems and benchmarks by their primary target, parallel unit, interaction interface, and evaluation emphasis.

III. PARAGUIBENCH

ParaGUIBench consists of three components: a parallel-native multi-device infrastructure with a shared file system, a curated dataset of 233 tasks spanning six categories under two domains (each also annotated by its parallel-execution pattern), and an evaluation system that jointly measures task success and parallel efficiency.

A. Parallel-Native Infrastructure

To isolate GUI control while preserving file-level coordination, each worker runs in a separate Docker container with its own full Ubuntu desktop, while all containers mount a shared directory for exchanging intermediate files. Separate desktops give each worker its own mouse cursor and keyboard focus, so concurrent workers cannot disrupt one another’s clicks and keystrokes, whereas the shared directory preserves the artifacts needed for coordination. The infrastructure runs up to N parallel workers ($N=5$ by default). A multi-container execution manager replaces OSWorld’s single virtual-machine backend. The infrastructure supports a configurable number of workers and configurable per-container resources, and restores every container to a clean state before each run.

B. Task Construction

We organize task construction into four parts: defining task categories, annotating parallel-execution patterns, collecting tasks from complementary sources, and applying quality-control filters.

1) *Task Categories*: The tasks span six task categories grouped under two application domains, summarized in Table II. We choose these categories to cover both information retrieval and operation & manipulation across browsers, office applications, and the file system.

Information retrieval (77 tasks) asks agents to navigate digital environments and produce a textual answer. We verify each answer by text-based matching. It covers web search in the browser and file search over Word, PowerPoint, and other document files.

Operation & manipulation (156 tasks) requires the agent to directly modify the environment state. We verify whether the task is completed by inspecting the application configurations or the file system. The operation & manipulation domain covers online shopping, file operation, web navigate, and search & write tasks. Table II lists each task category, its internal sub-categories, and one representative task instruction, illustrating the diversity of GUI interactions in ParaGUIBench.

2) *Parallel-Pattern Classes*: We characterize each task by the dependency structure of the trajectory that solves it, using three classes that we carry through to our results: *parallel_independent*, *parallel_dependent*, and *serial_parallel_independent* tasks decompose into sub-tasks with no inter-dependency: workers share no state and need no cross-worker communication, so all sub-tasks run concurrently (e.g., querying several independent stores, or assigning one file per worker). *parallel_dependent* tasks decompose into a directed acyclic graph (DAG) of sub-tasks: some sub-tasks consume the outputs of earlier ones, yet independent branches still run in parallel (e.g., a multi-hop query that splits into parallel sub-queries and then merges them). *serial* tasks admit no exploitable parallelism; because the dependency chain is unavoidable, the planner concentrates dispatch on essentially a single worker, driving the parallelism degree down toward 1.

3) *Task Sources*: We combine two task sources for complementary purposes. Existing benchmarks provide realistic applications, stable environments, and reusable evaluation

TABLE II
TASK DATASET STATISTICS WITH REPRESENTATIVE EXAMPLES. # MEANS THE NUMBER OF TASKS.

Domain	Category	#	Example Task Instruction
Info. Retrieval	web search (Statistic / Visual / Conditional / Multi-hop)	65	<i>How many Science magazines were published in 2024 that feature a fish on their cover?</i>
	file search (PPT / Word / generic)	12	<i>There are some files in the folder. Which AI-related papers have cartoon-style illustrations?</i>
Oper. Manip.	online shopping — WebMall (11 sub-categories)	91	<i>Find the cheapest offer for the Samsung Galaxy S24 Plus. If multiple offers share the lowest price, return all of them.</i>
	file operation (Combination / Batch (Word/Excel/PPT) / Coding)	42	<i>The folder contains many Word, PPT, and Excel files. Create different subfolders based on themes and content, and classify them into the subfolders.</i>
	web navigate (Bookmark / Settings)	13	<i>Navigate to Tesla’s official website, open the pages for Model Y, Model 3, and Model S for comparison, and add each page to the bookmarks.</i>
	search & write (File / Web)	10	<i>Please help me fill in the top five schools in the 2025 QS rankings and their detailed information into this table.</i>
Total		233	

infrastructure, whereas manually constructed tasks fill gaps in long-horizon decomposition and cross-worker coordination. Specifically, we adapt existing benchmarks and environments as the first source: From VeriWeb [33], we collect information-retrieval tasks with deterministic answers. From OSWorld [10], we combine several simple tasks into a single long-horizon task that decomposes into parallel sub-tasks. From WebMall [34], we reformulate transient-state tasks into checkpoint-verifiable ones (e.g., “reach a fixed product page” becomes “bookmark the target URL”); the verification protocol is detailed in Section III-C. The second source is manually constructed tasks for parallel execution and cross-worker coordination. They span each task category except online shopping, which is built entirely from the WebMall environment.

4) *Quality Control*: Reliable automated evaluation requires each task to have a unique, stable target and to demand genuine GUI interaction. We therefore apply two filters. The uniqueness filter rewrites or removes tasks with multiple valid answers or unstable targets. The shortcut filter removes tasks that an LMM can solve without genuine GUI interaction; human annotators validate candidates and inspect trial-run logs before inclusion.

C. Evaluation System

ParaGUIBench measures task success rate and agent efficiency. Efficiency comprises three metrics: the step reduction ratio, the parallelism degree, and token cost.

a) *Success Rate (SR)*: SR is the fraction of tasks solved correctly under the verification protocol.

b) *Step Reduction Ratio (S)*: The step reduction ratio measures how much parallel execution shortens the longest chain of sequential steps. Let $\tau_{\text{serial}} = \{(o_t, a_t)\}_{t=1}^T$ denote the trajectory of a single-worker GUI-only agent that runs the same task sequentially. The baseline length is

$$L_{\text{serial}} = |\tau_{\text{serial}}| = T,$$

where T is the number of perception–decision–action cycles executed by this agent. This trajectory is regarded as the serial

baseline. Since we report S against more than one such baseline agent (the worker-equivalent Seed-1.8 and the strongest serial agent, Claude Sonnet 4.6), each table states which baseline its S is computed against. A parallel run executes in R sequential planner rounds. Round r dispatches K_r workers. Worker k in round r runs $s_{r,k}$ steps. We define the parallel length L_{parallel} and the step reduction ratio S as

$$L_{\text{parallel}} = \sum_{r=1}^R \max_{k \in \{1, \dots, K_r\}} s_{r,k}, \quad (1)$$

$$S \triangleq \frac{L_{\text{serial}}}{L_{\text{parallel}}}.$$

Here $S > 1$ means parallel execution uses fewer critical-path steps than the serial baseline, while $S < 1$ means it uses more. Within each round, workers execute concurrently, so the round length is the largest per-worker step count, $\max_k s_{r,k}$. Summing these round lengths gives L_{parallel} , which we call the *critical-path steps*; it counts only worker GUI steps and excludes planner reasoning and aggregation calls. This is the *steps* column reported in all result tables. S contrasts *two different agent systems* (a single-agent GUI-only baseline vs. the planner–worker ParaGUI) on the same task. We report S at the corpus level, as the ratio of the mean L_{serial} to the mean L_{parallel} over all tasks—a task-weighted aggregate consistent with the steps in all result tables and with the pooling used for P .

c) *Parallelism (P)*: The parallelism degree quantifies how effectively the planner decomposes a task. Let the total number of worker steps actually expended be

$$L_{\text{total}} = \sum_{r=1}^R \sum_{k=1}^{K_r} s_{r,k}.$$

We define $P \triangleq L_{\text{total}}/L_{\text{parallel}}$. A value of $P=1.0$ indicates fully serialized dispatch, while $P=N$ indicates ideal N -way parallelism, upper-bounded by the per-task worker count. Unless stated otherwise, the per-task step counts we report

for ParaGUI are *critical-path steps* (L_{parallel}), not total worker steps (L_{total}).

d) Token Cost: We sum the token cost of the planner and all workers. This captures the resource overhead from parallelism. We count every token in full, with no prompt-cache savings.

e) Evaluation Procedures: We tailor the verification protocol to each task type and read the verdict from the real environment state rather than the agent’s self-report. We mark a task solved only when the agent recovers all expected items with nothing extraneous. We check information-retrieval answers by normalization and fuzzy matching: punctuation/format normalization, keyword coverage, numeric tolerance, and multi-value F1. This lets correct answers in different surface forms still pass. For web navigate and search-style online shopping tasks, verifying transient page state is hard. We sidestep this by reformulating “reach page X” as “bookmark page X”. The agent saves the target page(s); the evaluator reads the browser’s bookmark file and matches saved URLs against the expected targets. For cart and checkout shopping tasks, we verify from the live accessibility tree (the shopping cart and the order-confirmation page). We inspect file-operation outputs directly (cell values, formatting, document structure). We score search & write tasks cell-by-cell against a gold spreadsheet.

IV. PARAGUI

Figure 2 shows the planner-worker architecture of ParaGUI. ParaGUI adapts this pattern to parallel GUI execution through three design choices: adaptive round-based planning, self-contained dispatch, and disjoint-region assignment. Together, these let the planner revise later dispatches from intermediate outputs, write into each instruction the earlier results its sub-task depends on, and reduce conflicts over shared artifacts. Each worker executes its assigned instruction in an isolated Docker container.

A. Round-based Execution Workflow

ParaGUI’s planner exploits the parallel tool-calling of modern LMMs: in a single inference step it emits a batch of parallel invocations, each dispatching one sub-task to one worker. ParaGUI runs a round-based loop. The planner alternates between *reasoning* steps and synchronous parallel worker dispatch. The loop starts with an empty interaction history H and round counter $r=0$. In each round r , the planner conditions on the user instruction and the accumulated history H , then emits a batch of parallel invocations. Each invocation specifies the target worker index i , a natural-language sub-task g_i , and a set of dependency references D_i that point to the outputs of earlier rounds.

Rather than chaining sub-tasks into a sequence, ParaGUI emits self-contained sub-tasks that can be dispatched concurrently. We call this dispatch protocol *self-contained dispatch* (SCD). Each worker can see only the single instruction it is handed—not the planner’s reasoning, and not what the other workers did—so any result a sub-task needs from an earlier round must be written into that instruction before the worker starts. Each invocation targets one selected worker and carries

a natural-language sub-task together with a set of symbolic references to the outputs of earlier rounds. The planner marks each reference inline in its thought with a lightweight tag (e.g. `<dep round="3" agent="2"/>`); a regex parses these tags into the dependency set D_i , and when parsing fails, a heuristic analyzer reconstructs D_i from the wording of the sub-task. Before dispatch, the planner resolves every reference to a concrete string or file path and splices it into the sub-task, turning it into a self-contained instruction with no unresolved references. The worker then executes it on its own, never reading the planner’s history or any other worker’s state.

All invocations in a round execute concurrently, one per Docker container. The planner blocks until every worker returns. Each worker returns a textual summary along with the paths of any files it produced. The planner does *not* receive worker screenshots, which keeps its context length bounded as N scales. We append these summaries to H and start the next round for the planner.

After each round, ParaGUI checks two termination conditions. First, the planner can emit a `<answer>...` tag in its thought; its payload becomes the task output. Second, the global GUI-step budget runs out, after which no further worker dispatch is allowed and a synthetic “budget exhausted” message forces the planner to commit a final answer in the next round. In either case, the evaluation protocol scores the final environment state regardless of whether the agent produced an explicit answer.

B. Planner

Each round, the planner decides the next dispatch, mitigates conflicts over shared artifacts, and aggregates the returned results. First, it identifies sub-tasks that are independent and safe to dispatch concurrently, while deferring those that depend on the current round’s outputs. Unlike static DAG planners [18], [20], which fix the dependency graph upfront, ParaGUI revises later dispatches from intermediate worker summaries. The planner may also re-target the same worker when preserving its UI state is useful. Second, when multiple sub-tasks touch a shared artifact, the planner assigns non-overlapping responsibilities (*disjoint-region assignment*) where possible and serializes operations that genuinely conflict; these decisions come from the planner LMM over the accumulated history H , with no hand-coded resource manager. Third, after all workers return, the planner aggregates their textual summaries and produced file paths, updates the interaction history, and decides whether to dispatch another round or terminate.

C. Worker

ParaGUI treats each worker as an interchangeable GUI executor rather than introducing a specialized worker policy. Each worker controls a single Docker container and shares the mounted file system with the others. Workers reuse the OSWorld primitive action space [10] (click, scroll, type, hotkey, ...). This lets us swap in any vision-capable LMM as a worker backbone. On invocation, a worker receives the natural-language sub-task g_i with the dependency references D_i already

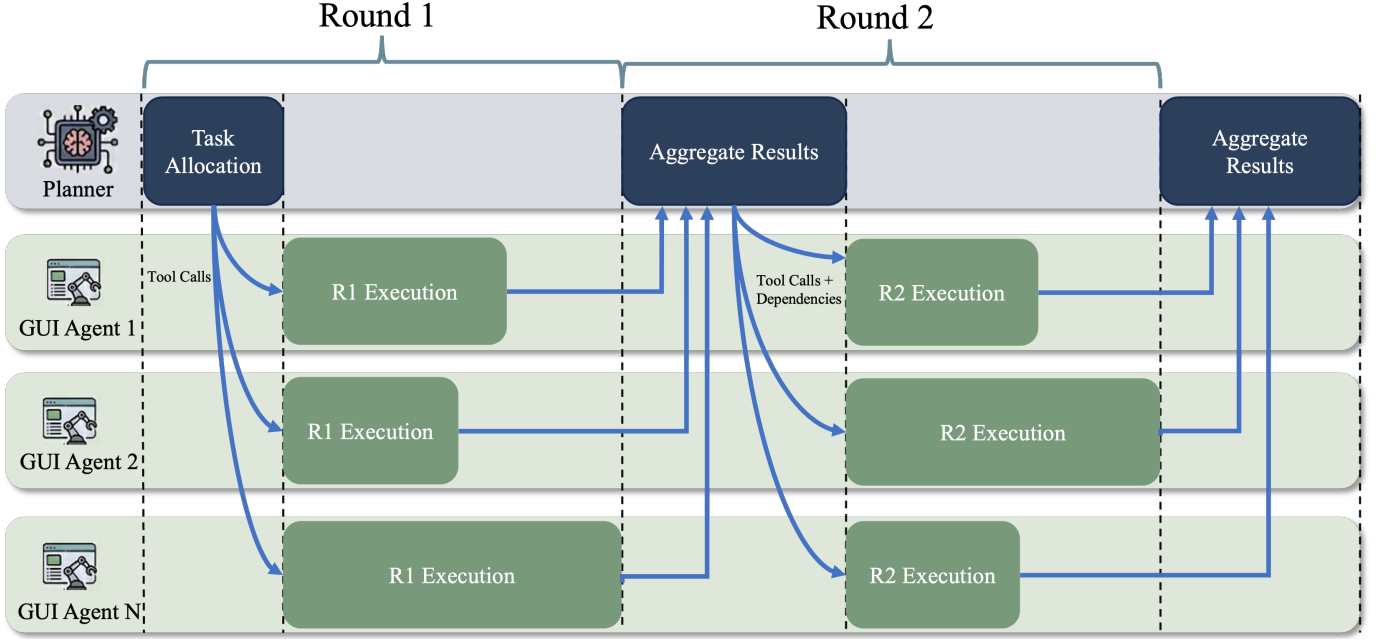


Fig. 2. The ParaGUI planner-worker architecture. Each round, the planner dispatches a batch of sub-tasks to up to N GUI workers running in parallel, aggregates their returned summaries, and decides whether to dispatch another round or terminate.

resolved to concrete strings or file paths. The worker then returns a structured summary: a textual report of its actions and the paths of any files produced. Cross-worker communication uses only the shared directory `/home/user/shared/`. We do not support direct messaging between workers.

a) *Step budgets and explicit failure signals*: Each worker invocation runs at most 25 perception–decision–action steps. The global GUI-step budget across all workers in one task is 200 steps. ParaGUI propagates two explicit failure signals to the planner: (i) a *step-cap overflow*, when a worker reaches its per-invocation step cap and returns `FAIL`; and (ii) a *voluntary failure*, when a worker invokes `fail()` on an unrecoverable UI state. The per-invocation cap of 25 steps balances sub-task completion against token cost: a larger budget lets a stuck worker accumulate redundant actions, and thus spend more tokens, while a smaller budget leaves too little room to complete a non-trivial sub-task. By contrast, hitting the global cap injects a synthetic “budget exhausted” message that forces the planner to commit a final answer in the next round.

V. EXPERIMENTS AND ANALYSIS

We organize our experiments around five research questions. **RQ1**: How does ParaGUI compare with serial GUI agents in task success and efficiency? **RQ2**: Under what task conditions is parallel GUI execution beneficial? **RQ3**: What failure modes arise when ParaGUI runs on ParaGUIBench? **RQ4**: Which components and configurations govern ParaGUI’s capability–efficiency trade-off? **RQ5**: Does the planner–worker architecture transfer to standard serial GUI tasks? RQ1 reports the full 233-task comparison together with a paired analysis on the *agree-subset*—tasks on which ParaGUI and a baseline have the same outcome (both correct or both fail); RQ2 and RQ3

analyze the full ParaGUIBench; RQ4 uses a 51-task diagnostic subset; and RQ5 evaluates the full 369-task OSWorld suite with $N=1$.

A. Experimental Setup

a) *Models*: We evaluate a set of representative LMMs across the planner–worker architecture. ParaGUI uses GPT-5.4 [35] as its planner backbone and Doubao Seed-1.8 [36] (Seed-1.8 hereafter) as its worker backbone. This pairing combines strong long-context reasoning on the planner side with broad action-space coverage and lower per-step cost on the worker side. As serial GUI-only baselines, we evaluate four single GUI agents (no planner), each driven by one LMM backbone and competitive among publicly reported GUI agents on OSWorld and related benchmarks at submission time. The four are Seed-1.8, Qwen3-VL-235B-A22B-Instruct [37], Holo3-35B-A3B [38], and Claude Sonnet 4.6 [39]. Together they span general-purpose and GUI-specialized model families.

b) *Execution Configurations*: We evaluate two kinds of systems. The four serial baselines run in *GUI-only* mode: a single GUI agent executing one perception–decision–action loop per step, with no planner. ParaGUI runs in *Parallel* mode with $N=5$ workers concurrently on separate isolated Docker containers, each allocated 4 GiB of RAM and 2 vCPUs. Each serial GUI-only baseline runs a single agent in one such container under the same per-container resources. The planner coordinates the workers and does not occupy a Docker container. Ablations also vary the planner and worker backbones and the parallelism degree N . Unless otherwise noted, every agent is decoded with temperature 0 and a fixed random seed of 42 for reproducibility.

TABLE III
MAIN RESULTS ON PARAGUIBENCH (233 TASKS). PARAGUI ($N=5$) VS. FOUR SERIAL GUI-ONLY BASELINES. SR: SUCCESS RATE; STEPS: AVG. CRITICAL-PATH STEPS; TOKENS: AVG. PER-TASK TOKEN USAGE (M). $\uparrow/\downarrow$: HIGHER/LOWER IS BETTER.

Domain	Category	#	SR % (avg) $\uparrow$					Critical path (avg) $\downarrow$					Tokens (avg, M) $\downarrow$				
			ParaGUI	Seed-1.8	Qwen3-VL	Holo3	Claude 4.6	ParaGUI	Seed-1.8	Qwen3-VL	Holo3	Claude 4.6	ParaGUI	Seed-1.8	Qwen3-VL	Holo3	Claude 4.6
Info. Retrieval	web search	65	41.5	24.6	12.3	6.1	32.3	38.9	35.9	47.4	125.1	87.3	1.07	0.38	0.58	3.15	2.48
	file search	12	75.0	8.3	16.7	33.3	33.3	20.4	12.6	36.1	44.5	74.7	0.23	0.10	0.41	0.90	0.45
Operation	online shopping	91	54.9	38.5	9.9	2.2	44.0	32.7	37.8	48.0	148.9	77.6	0.69	0.38	0.59	3.65	2.17
	file operation	42	33.3	21.4	11.9	14.3	21.4	54.5	44.1	64.8	108.3	72.8	0.95	0.49	0.84	3.06	2.09
	web navigate	13	53.8	23.1	15.4	0.0	23.1	29.2	28.8	72.9	149.6	39.5	0.44	0.28	0.93	3.19	0.66
	search & write	10	10.0	10.0	0.0	0.0	10.0	59.2	39.8	75.3	81.2	48.0	0.87	0.46	1.01	2.20	1.22
Total		233	46.4	27.9	11.2	6.9	33.5	38.7	36.7	52.8	126.7	75.9	0.81	0.38	0.66	3.18	2.03

c) *Visual-History Configuration*: Each agent runs under its officially recommended visual-history retention policy: Seed-1.8 keeps the most recent 3 screenshots, Claude Sonnet 4.6 retains up to 10 with adaptive sizing, and Qwen3-VL and Holo3 each keep the most recent 5 screenshots. ParaGUI workers (Seed-1.8 backbone) inherit Seed-1.8’s 3-screenshot policy. The GPT-5.4 planner consumes text-only worker summaries and receives no screenshots, so its token consumption does not scale with worker visual context. Token gaps reported in Tables III to V therefore reflect both intrinsic per-call cost and these visual-history configuration differences. We report numbers under each model’s actual deployment configuration rather than normalize across agents, because (a) official configurations are how each model is actually deployed in practice and (b) any normalization would itself introduce a hidden axis of comparison. An additional ablation restricts Claude Sonnet 4.6 to a 3-screenshot budget and shows that the token gap does *not* narrow under this normalization—in fact, the restricted Claude configuration spends more steps and more tokens at a slightly lower success rate (full numbers in Table XII).

d) *Evaluation Split*: Vision-intensive multi-agent runs are expensive, so we adopt a two-tier evaluation. Our main evaluation reports each method on the full 233-task benchmark under its default configuration. For sweeping many planner and worker combinations and fine-grained ablation studies (e.g. planner vs. worker impact), we use a 51-task subset (file search 7, web search 10, online shopping 20, web navigate 2, file operation 9, search & write 3). This subset is selected to approximate the benchmark’s domain mix while keeping compute tractable.

B. RQ1: Serial-Agent Comparison

On the full 233-task benchmark, ParaGUI achieves a higher success rate than the strongest serial baseline (46.4% vs. 33.5%) while using roughly half as many critical-path steps and fewer than half as many tokens. The full-benchmark comparison establishes these success-rate gains; at matched outcomes, the paired analyses then show that ParaGUI uses more tokens and does not consistently reduce steps relative to Seed-1.8, but uses fewer steps and tokens than Claude Sonnet 4.6.

We compare ParaGUI (Planner+Worker, $N=5$) against four representative serial GUI-only baselines on the full ParaGUIBench (233 tasks; baselines introduced in Section V-A). Table III reports category-level SR, critical-path steps, and tokens, grouped by domain.

a) *Information Retrieval*: On the information-retrieval domain (Table III, 77 tasks), ParaGUI outperforms the strongest serial baseline Claude Sonnet 4.6 on both categories: web search (41.5% vs. 32.3%) and file search (75.0% vs. 33.3%). It does so with far fewer critical-path steps (web search 38.9 vs. 87.3; file search 20.4 vs. 74.7) and fewer tokens (web search 1.07M vs. 2.48M; file search 0.23M vs. 0.45M).

b) *Operation & Manipulation*: On the operation domain, ParaGUI outperforms Claude Sonnet 4.6 on the three categories whose sub-tasks can run on independent workers: online shopping (54.9% vs. 44.0%, +10.9 points), file operation (33.3% vs. 21.4%, +11.9 points), and web navigate (53.8% vs. 23.1%, +30.7 points). Search & write, whose sub-tasks must edit one shared document, remains the hardest category for both systems (10.0% vs. 10.0%).

c) *Efficiency and Cost*: Against the strongest serial baseline Claude Sonnet 4.6, ParaGUI attains higher SR (46.4% vs. 33.5%, +12.9 points). It uses roughly half the critical-path steps (38.7 vs. 75.9) and less than half the tokens (0.81M vs. 2.03M) under each model’s default visual-history configuration. Against Seed-1.8—a token-efficient baseline that is also our worker backbone—ParaGUI uses comparable steps (38.7 vs. 36.7) and delivers a +18.5-point SR gain (46.4% vs. 27.9%). ParaGUI spends about $2.1\times$ as many tokens as Seed-1.8 (0.81M vs. 0.38M); this overhead is partly explained by a baseline behavior pattern, in which Seed-1.8 frequently terminates early on incorrect trajectories with high confidence, well before the step budget (Table VIII), lowering its step and token counts without improving its success rate. Because Seed-1.8 is the worker in both systems, the +18.5-point gain cannot be attributed to a stronger worker backbone; it must arise from the planner-worker architecture, including decomposition, SCD, and parallel dispatch. To separate efficiency from accuracy, we next restrict the comparison to the *agree-subset*: tasks on which ParaGUI and the baseline have the same outcome (both correct or both fail). We focus this analysis on Seed-1.8 (170/233 paired tasks; Table IV)—the token-efficient worker backbone—since ParaGUI already outperforms Claude Sonnet 4.6 on both success rate and efficiency (Table V); the complementary Claude Sonnet 4.6 comparison appears there.

Table IV reports a breakdown of the agree-subset by partition (both-correct vs. both-fail) and category. To keep each row well populated, it shows only categories with ≥ 10 paired tasks; the remaining 31 paired tasks fall in categories with fewer than 10 each.

Table IV (paired vs. Seed-1.8) highlights a clear category-level split in the *both-correct* partition. On online shopping,

TABLE IV

EFFICIENCY ON THE PAIRED AGREE-SUBSET (170/233 TASKS): PARAGUI ($N=5$) VS. THE SERIAL SEED-1.8 BASELINE (ITS WORKER BACKBONE), FOR CATEGORIES WITH ≥ 10 PAIRED TASKS. USING THE SAME SEED-1.8 GUI EXECUTOR CONTROLS FOR WORKER-BACKBONE STRENGTH AND ISOLATES THE EFFECT OF THE PLANNER-WORKER ARCHITECTURE. *Step Red.*: BASELINE STEPS $\div$ PARAGUI STEPS ($\uparrow$, >1 MEANS PARAGUI USES FEWER STEPS). *Token $\times$* : PARAGUI TOKENS $\div$ BASELINE TOKENS ($\downarrow$, <1 MEANS PARAGUI IS CHEAPER). THE CRITICAL-PATH COLUMN LISTS PARAGUI AND SEED-1.8 MEAN STEPS SIDE BY SIDE, NOT A RATIO.

Partition	Category	#	Critical path (Para vs. Seed)	Step Red. (S) $\uparrow$	Token $\times$ $\downarrow$
Both Correct	online shopping	29	31.7/43.1	1.36$\times$	1.12 $\times$
	web search	13	31.1/25.6	0.82 $\times$	4.35 $\times$
Both Fail	online shopping	35	38.8/37.1	0.96 $\times$	2.46 $\times$
	web search	34	43.1/41.3	0.96 $\times$	2.62 $\times$
	file operation	28	52.9/43.1	0.82 $\times$	2.04 $\times$

TABLE V

EFFICIENCY ON THE PAIRED AGREE-SUBSET VS. THE CLAUDE SONNET 4.6 BASELINE, FOR CATEGORIES WITH ≥ 10 PAIRED TASKS; COMPLEMENTARY TO TABLE IV. *Step Red.*: BASELINE STEPS $\div$ PARAGUI STEPS ($\uparrow$, >1 MEANS PARAGUI USES FEWER STEPS). *Token $\times$* : PARAGUI TOKENS $\div$ BASELINE TOKENS ($\downarrow$, <1 MEANS PARAGUI IS CHEAPER). ROWS SHOWN COVER 129 PAIRED TASKS; THE REMAINING PAIRED TASKS FALL IN CATEGORIES WITH FEWER THAN 10 EACH AND ARE OMITTED. THE CRITICAL-PATH COLUMN LISTS PARAGUI AND CLAUDE SONNET 4.6 MEAN STEPS SIDE BY SIDE, NOT A RATIO.

Partition	Category	#	Critical path (Para vs. Claude)	Step Red. (S) $\uparrow$	Token $\times$ $\downarrow$
Both Correct	online shopping	27	23.7/74.2	3.12$\times$	0.26$\times$
	web search	15	24.3/65.5	2.69 $\times$	0.36 $\times$
Both Fail	online shopping	28	41.9/94.8	2.26 $\times$	0.36 $\times$
	web search	32	42.4/101.3	2.39 $\times$	0.38 $\times$
	file operation	27	54.3/74.4	1.37 $\times$	0.46 $\times$

ParaGUI shortens the critical path by 1.36 $\times$ at a small token overhead (1.12 $\times$): parallel dispatch adds only a modest token cost when independent stores can be assigned to separate workers. On web search, the already-short Seed-1.8 trajectory leaves little room for step reduction; ParaGUI in fact uses about 22% more steps ($S=0.82\times$); because self-contained dispatch (SCD) rewrites the full instruction and resolved dependencies into each worker’s sub-task every round, the token cost inflates to 4.35 $\times$. In the *both-fail* partition, step counts are roughly even—ParaGUI requires marginally more critical-path steps (0.82–0.96 $\times$)—but token costs are uniformly 2.0–2.6 $\times$ higher. Because Seed-1.8 workers commit confidently to wrong answers (Table VIII), the planner receives no signal that the trajectory has reached a dead end and continues dispatching parallel branches.

Table V (paired vs. Claude Sonnet 4.6) presents the complementary result. In both partitions, ParaGUI uses both fewer steps and fewer tokens: 2.7–3.1 $\times$ fewer steps with 0.26–0.36 $\times$ tokens on *both-correct*, and 1.4–2.4 $\times$ fewer steps with 0.36–0.46 $\times$ tokens on *both-fail*. The largest single-category gain is on both-correct online shopping (3.12 $\times$ fewer steps at 0.26 $\times$ tokens). The two paired tables thus reveal a consistent pattern: against Seed-1.8 (the same worker backbone), ParaGUI trades extra tokens for higher success rate rather than for speed; against the token-heavier Claude Sonnet 4.6, it achieves both higher success rate and lower token usage.

C. RQ2: When Parallel Execution Helps

Exploitable parallelism is governed primarily by task dependency structure rather than application domain alone:

decomposable tasks reach a parallelism degree of approximately 2.2, whereas serial tasks remain near 1. Shared-document conflicts, already-short baseline trajectories, and dependence on visual context reduce the resulting efficiency gains.

We measure exploitable parallelism by the parallelism degree $P = L_{\text{total}}/L_{\text{parallel}}$ —how much shorter the critical path is than running the same decomposed sub-tasks serially. A task’s intrinsic decomposability, rather than its application domain alone, primarily governs P : the overall task-weighted degree is $P=2.13$, with per-category values in Table VI.

Figure 3 traces a successful five-worker parallel run on one decomposable task. Grouping tasks by their dependency structure isolates where parallelism yields gains. Table VII reports SR, critical-path steps, and parallelism degree for the three parallel-pattern classes. The planner reduces parallelism to $P\approx 1.1$ on *serial* tasks and reaches $P=2.19$ and $P=2.28$ on the two decomposable (parallel-pattern) classes; this low serial value shows that round-based dispatch avoids over-dispatching when no parallelism is available. Meanwhile, SR remains similar across the three classes (45.7–48.0%), suggesting that the higher parallelism on decomposable tasks is not associated with lower task success.

The categories benefit through different mechanisms. The planner splits a web-search query into sub-queries that run on separate workers in parallel, and online shopping decomposes similarly by assigning each store to a different worker. File search instead benefits from giving each worker a single file, which removes the cross-file interference that degrades the single-agent baseline. The gains are eroded where parallelism cannot help: *visual* web-search answers depend on page content that a text-only sub-task cannot convey, and search & write stays hardest for both systems because workers must coordinate disjoint edits on one shared document; this contention also lengthens ParaGUI’s critical path (59.2 vs. 48.0 steps for Claude Sonnet 4.6).

D. RQ3: Failure Modes

Most ParaGUI failures are silent miscompletions in which the system reports completion even though the final environment state is incorrect. Among the 111 silent miscompletions, single-worker GUI execution is the largest root cause (53% of the 111), while planner decomposition and aggregation together account for 30%. Figure 4 shows one such silent miscompletion: the worker recovers from a visual-grounding error during execution, yet the file it finally saves still fails the rule-based check. We now analyze the full failure set quantitatively. We manually assign a primary root cause to each of ParaGUI’s 125 failed tasks ($N=5$, full ParaGUIBench) by inspecting the execution trajectories case by case. The 125 failures fall into two groups: 111 silent miscompletions (89%), where the system emits a substantive final answer and self-reports success while the true environment state is wrong, and 14 honest terminations (11%) via a voluntary `fail()` call or global budget exhaustion. Explicit termination therefore accounts for only a small minority of failures.

We sort the 111 miscompletions into four root causes (shares of the 111 below). Single-worker GUI execution is the largest,

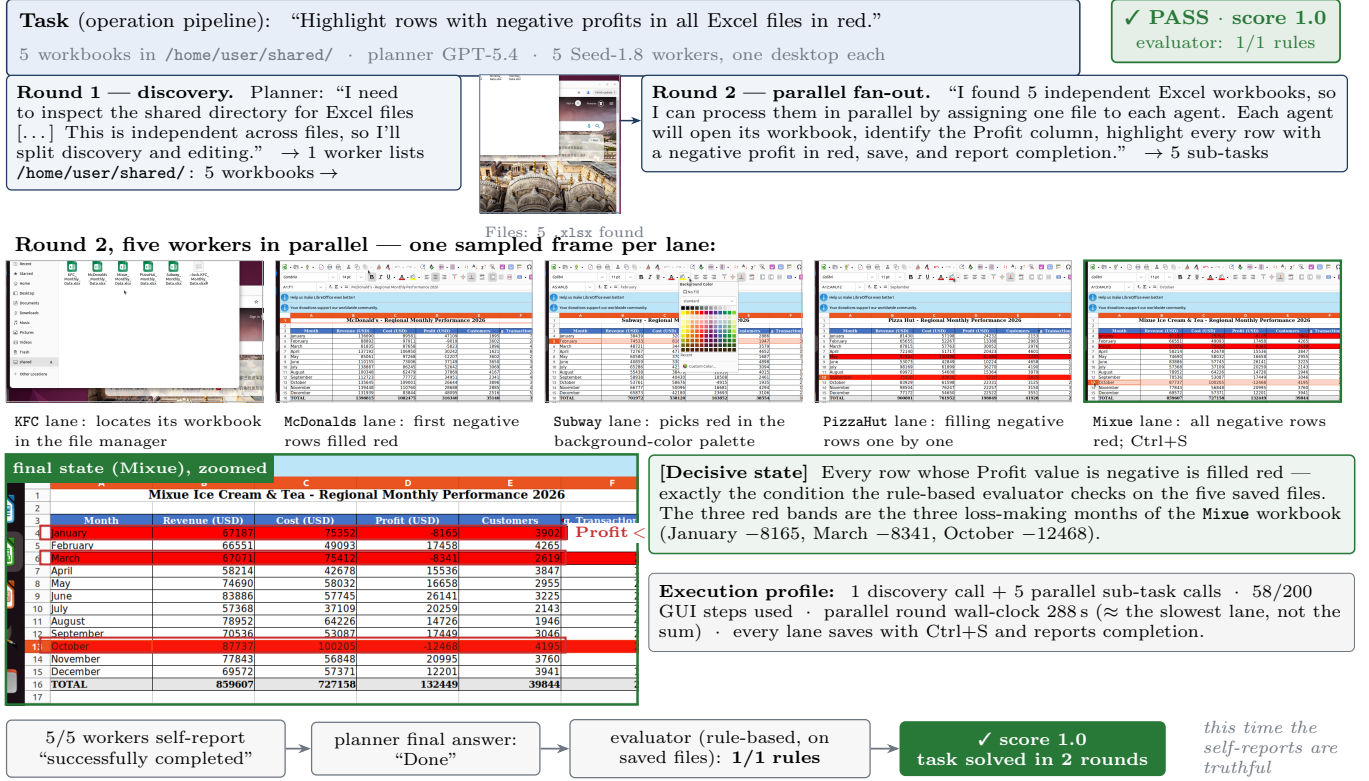


Fig. 3. A successful ParaGUI trace on ParaGUIBench. *Round 1*: one worker inspects the shared folder and reports five workbooks. *Round 2*: the planner gives each of five workers one workbook; they run concurrently, so the round costs the most steps any single worker takes, not the sum. *Zoom*: the final state—every negative-profit month filled red—is exactly what the rule-based evaluator checks (1/1 rules, score 1.0).

TABLE VI

PER-CATEGORY PARALLELISM ON THE FULL PARAGUIBENCH (233 TASKS). *Worker steps* (L_{total}) AND *Critical path* (L_{parallel}) ARE PER-TASK AVERAGES; THE LATTER MATCHES THE *steps* COLUMN OF TABLE III. *Para.* IS THE PARALLELISM DEGREE $P = L_{\text{total}}/L_{\text{parallel}}$ (TASK-WEIGHTED). BOLD MARKS THE PER-COLUMN MAXIMUM.

Domain	Category	#	Worker steps (avg)	Critical path (avg)	Para.
Info. Retrieval	Web Search	65	108.0	38.9	2.78
	File Search	12	24.7	20.4	1.21
Operation	Online Shopping	91	71.4	32.7	2.18
	File Operation	42	92.1	54.5	1.69
	Web Navigate	13	47.2	29.2	1.61
	Search & Write	10	87.1	59.2	1.47
Total		233	82.2	38.7	2.13

TABLE VII

PARAGUI RESULTS ON PARAGUIBENCH (233 TASKS) BY PARALLEL-PATTERN CLASS. *parallel_independent*: SUB-TASKS WITH NO INTER-DEPENDENCY; *parallel_dependent*: SUB-TASK DAG; *serial*: NO EXPLOITABLE PARALLELISM. SR: SUCCESS RATE; PARALLELISM = WORKER STEPS ÷ CRITICAL-PATH STEPS.

Parallel-pattern class	#	SR (%)	Critical path (avg)	Worker steps (avg)	Para.	Tokens (M)
parallel_independent	164	45.73	33.97	74.45	2.19	0.73
parallel_dependent	50	48.00	53.80	122.70	2.28	1.22
serial	19	47.37	39.58	43.05	1.09	0.43
Total	233	46.35	38.68	82.24	2.13	0.81

at 59 cases (53%): a worker gets an in-principle-solvable sub-task wrong (misread values; a search so literal that it reports an existing item as “not found”; wrong visual judgment; mis-clicks). Planner errors form the second group (together 30%):

20 *decomposition* errors (18%) and 13 *aggregation* errors (12%). A decomposition error partitions the task badly—for example, instructing each WebMall store to bookmark its own local cheapest item with no global comparison, or splitting a shared

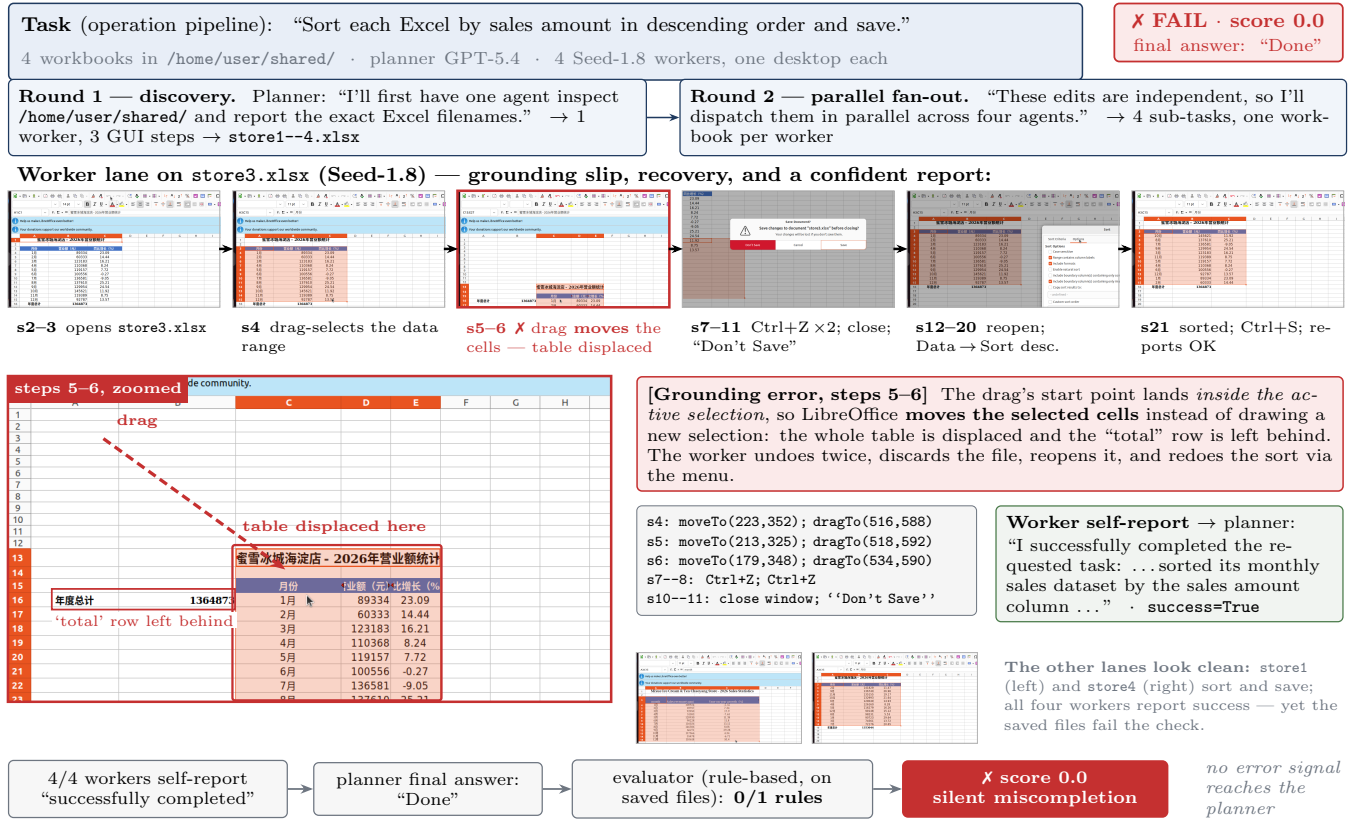


Fig. 4. A silent miscompletion on ParaGUIBench. The planner decomposes correctly, and the store3 worker even recovers from a grounding slip (a drag inside the active selection *moves* the cells instead of selecting them; zoom). Yet all four workers report success, the planner answers “Done”, and the rule-based evaluator rejects the saved files (0/1 rules, score 0.0)—no error signal ever reaches the planner.

document so that workers overwrite each other (consistent with search & write being the weakest category; Table XI). An aggregation error instead merges correct per-worker results into a wrong final answer. The remaining 19 (17%) are premature success claims, distinct from the 14 honest terminations above because here the worker wrongly reports success on an unsolved task.

The over-confident “task-complete” signal is therefore a symptom rather than a root cause in its own right: in decreasing order of frequency, it stems from single-worker GUI execution, planner decomposition and aggregation, and premature success claims. It also sharpens how to read Table VIII: most failures are confident wrong answers rather than early give-ups (honest fail() calls).

Table VIII reports the average step count on failed tasks, broken down by category. The main contrast is the gap between Seed-1.8 (36.6 avg. steps on fail) and Claude Sonnet 4.6 (80.1 avg. steps on fail). In Search & Write, Seed-1.8 is no longer an extreme early-stop outlier: its 40.9 average steps are close to Claude’s 47.1. Manual inspection of the underlying trajectories confirms that most early Seed-1.8 terminations are confident done(answer) actions that the evaluator rejects, rather than voluntary fail() calls; this is a *calibration* failure—the same over-confident task-complete pattern identified above—not an early give-up. Claude Sonnet 4.6 appears less prone to early confident stopping: on tasks it cannot solve, it spends *more* steps

TABLE VIII
AVERAGE CRITICAL-PATH STEPS ON *failed* TASKS BY CATEGORY, COMPUTED OVER EACH SYSTEM’S OWN SET OF FAILED TASKS (THE FAILED SETS DIFFER ACROSS SYSTEMS, AS THE #FAIL COLUMNS SHOW). EACH CELL: #FAILED TASKS AND MEAN STEP COUNT.

Category	Seed-1.8		ParaGUI		Claude 4.6	
	#fail	steps	#fail	steps	#fail	steps
Web Search	49	39.1	38	42.9	44	96.5
File Search	11	12.8	3	12.7	8	98.2
Online Shopping	56	36.1	41	35.5	51	82.8
File Operation	33	42.4	28	52.9	33	69.5
Web Navigate	10	31.0	6	34.8	10	45.1
Search & Write	9	40.9	9	59.9	9	47.1
Total	168	36.6	125	42.8	155	80.1

than its overall average (80.1 vs. 75.9) before failing. ParaGUI falls between the two baselines on failed tasks (42.8 steps): it inherits a Seed-1.8 worker but adds steps through planner-level re-dispatch. This re-dispatch gives additional opportunities for correction but does not verify the final environment state or detect silent miscompletions.

E. RQ4: Component and Scaling Analysis

On the 51-task diagnostic subset, SCD is a major structural contributor: removing it lowers success from 39.2% to 27.5%.

The worker backbone sets the capability ceiling, while increasing the worker count N primarily shortens the critical path without monotonically improving success. This is consistent with the failure attribution above, where single-worker GUI execution was the largest root cause, although planner decomposition and aggregation still contribute a substantial share. To quantify the contribution of each component, we conduct ablation studies on the same 51-task subset (on which ParaGUI scores 39.2% SR, vs. 46.4% on the full 233 tasks). Table IX summarizes the results. We provide the full parallelism scaling analysis in the appendix.

a) *Component analysis*: The ablation isolates three findings. In Table IX, row labels follow a fixed convention: `plan-*` varies the planner while always using a Seed-1.8 worker, and `*-gui` varies the worker while always using a GPT-5.4 planner.

First, the gain is primarily associated with the planner’s decomposition and SCD, not the planner model. With a Seed-1.8 worker, both a GPT-5.4 planner and a Seed-1.8 planner (`plan-seed-1.8`) reach 39.2% success (a Kimi K2.5 [5] planner, `plan-kimi`, reaches 33.3%). All three far exceed the 17.6% Seed-1.8 serial baseline (its worker backbone). Disabling SCD (`-SCD`) drops the success rate to 27.5%. Among these Seed-1.8-worker planner variants, the GPT-5.4 planner is the most efficient: it reaches the top success rate in the fewest critical-path steps (33.3 vs. 40.8 for `plan-seed-1.8`).

Second, *the worker backbone sets the capability ceiling and governs efficiency*. A Claude Sonnet 4.6 worker (`claude-gui`) attains the highest SR (47.1%); Seed-1.8 reaches 39.2%, and Kimi K2.5 (`kimi-gui`) 35.3%. Seed-1.8 remains the most efficient choice (33.3 steps and 0.73M tokens, vs. 50.0 steps and 1.98M tokens for Claude) and gives the best accuracy–efficiency trade-off among the three worker variants.

Third, *N shifts which tasks are solved without monotonically improving capability*. Across $N=1, 3$, and 5 on the 51-task subset, 18 tasks are solved by all three configurations; $N=1$ uniquely solves 3 further tasks, while $N=3$ and $N=5$ each unlock 2 tasks that $N=1$ misses. Two opposing mechanisms are at play. First, extra parallel branches introduce dispatch errors, so the $N=3$ and $N=5$ configurations lose the three tasks that only $N=1$ solves. Second, the same branching unlocks decompositions that a single worker cannot complete within the 25-step per-invocation cap. Aggregate SR is therefore nearly constant and, if anything, slightly lower at larger N (41.2%, 39.2%, and 39.2% for $N=1, 3, 5$). Critical-path steps fall monotonically (60.5, 39.0, and 33.3, respectively), so the dominant effect of larger N on this subset is a shorter critical path at nearly unchanged accuracy.

F. RQ5: Transfer to OSWorld

On the full 369-task OSWorld suite, ParaGUI with $N=1$ improves overall SR from 51.5% to 59.1%, but its benefit is uneven: it matches the baseline on OS and Thunderbird and falls behind on Chrome and GIMP, where tasks demand stronger visual continuity than text-only sub-tasks preserve. To check whether our planner–worker architecture transfers beyond parallel-native tasks, we evaluate ParaGUI ($N=1$, serial) on

TABLE IX

ABLATION OF THE PLANNER AND WORKER ON THE 51-TASK SUBSET. SR: OVERALL SUCCESS RATE (ALL 51); STEPS: AVG. CRITICAL-PATH STEPS; TOKENS: AVG. PER-TASK TOKENS (M). UNLESS NOTED, PARALLEL VARIANTS USE A GPT-5.4 PLANNER AND SEED-1.8 WORKER; EACH VARIANT’S PLANNER AND WORKER PAIRING AND THE FULL PER-CATEGORY SR APPEAR IN TABLE XII. ROWS `PLAN-*` VARY THE PLANNER (WORKER FIXED TO SEED-1.8); ROWS `*-GUI` VARY THE WORKER (PLANNER FIXED TO GPT-5.4). `-SCD` REPLACES SCD WITH A NAIVE “CONTINUE/FINISH” DISPATCH. BOLD MARKS THE DEFAULT PARAGUI CONFIGURATION ($N=5$), NOT THE PER-COLUMN BEST. $\uparrow/\downarrow$: HIGHER/LOWER IS BETTER.

Configuration	SR (51) $\uparrow$	steps $\downarrow$	Tokens $\downarrow$
<i>Serial baseline (GUI-only, no planner)</i>			
Seed-1.8	17.6%	28.3	0.29
Claude Sonnet 4.6	35.3%	67.1	1.64
<i>Parallel variants</i>			
plan-seed-1.8	39.2%	40.8	0.74
plan-kimi	33.3%	45.6	0.88
kimi-gui	35.3%	54.0	1.37
claude-gui	47.1%	50.0	1.98
ParaGUI <code>-SCD</code> ($N=5$)	27.5%	30.6	0.55
ParaGUI ($N=1$)	41.2%	60.5	0.63
ParaGUI ($N=3$)	39.2%	39.0	0.71
ParaGUI ($N=5$, def.)	39.2%	33.3	0.73

the standard OSWorld benchmark [10] against Seed-1.8 in GUI-only mode. On the full 369-task suite, ParaGUI ($N=1$) reaches 59.1%, ahead of the Seed-1.8 GUI-only baseline at 51.5% (+7.6 points). This overall gain is not uniform: it reflects two opposing per-application effects (Table X), with ParaGUI matching or outperforming Seed-1.8 on every application except Chrome and GIMP.

The dividing line is not single-application vs. multi-application, but whether a task can be split into self-contained textual sub-tasks without losing the visual context that each step depends on. Office and tool-operation tasks (Calc, Impress, VLC, VS Code) decompose into bounded, independent steps that a stateless worker can execute from a one-line instruction (for example, “enter the specified formula in A1” or “set the title font size to 18 pt”); here the planner’s global decomposition is a net gain over a single agent that accumulates context errors on long trajectories. Chrome and GIMP are instead continuous exploratory flows in which each step depends on prior visual state. Decomposing them into stateless, text-only fragments removes the visual continuity that subsequent actions need: the worker reads a textual log of what happened but cannot see the screens that produced it, and the planner, which receives no screenshots, must infer the missing state from text alone.

This gap is unlikely to be closed by changes to the planner or the text interface alone; the evidence points to persisting each worker’s own visual frames across rounds as a likely missing factor. Overall, the planner–worker design transfers a net planning benefit to serial environments, conditional on how well each task can be expressed as text-only sub-tasks.

Per application (Table X), ParaGUI gains most on tool-operation and office tasks: LibreOffice Calc (+27.6 points), Impress (+23.4), VLC (+17.7), and VS Code (+13.0). Gains are smaller on the multi-application split (+8.9) and Writer (+4.4). ParaGUI ties on OS and Thunderbird and falls behind

TABLE X
OSWORLD RESULTS PER APPLICATION. PARAGUI ($N=1$)—A GPT-5.4 PLANNER COORDINATING A SINGLE SEED-1.8 WORKER (NO PARALLEL WORKER EXECUTION)—VS. THE SEED-1.8 GUI-ONLY BASELINE ON THE FULL 369-TASK SUITE. MISSING RESULTS COUNTED AS 0; SR IS BINARY (SUCCESS ONLY IF OSWORLD SCORE = 1.0). $\uparrow$: HIGHER IS BETTER.

Application	#	ParaGUI ($N=1$)	GUI-only (Seed-1.8)
		SR $\uparrow$	SR $\uparrow$
chrome	46	45.7%	63.0%
gimp	26	65.4%	80.8%
libreoffice_calc	47	78.7%	51.1%
libreoffice_impress	47	68.1%	44.7%
libreoffice_writer	23	60.9%	56.5%
multi_apps	101	30.7%	21.8%
os	24	75.0%	75.0%
thunderbird	15	86.7%	86.7%
vlc	17	82.4%	64.7%
vs_code	23	91.3%	78.3%
Overall	369	59.1%	51.5%

only on Chrome (−17.3) and GIMP (−15.4). One Chrome case illustrates a likely cause: the round-2 worker received a 4,387-character verbatim history of the prior round yet still failed, suggesting that textual history alone is insufficient and that visual continuity is the missing factor—a target for future work.

VI. CONCLUSION

We identified the sequential interaction paradigm as a key efficiency bottleneck for GUI agents on long-horizon complex tasks. We took a first step toward evaluating multi-agent parallel coordination in this setting. We introduced ParaGUIBench, a 233-task benchmark for parallel execution and coordination in realistic stateful GUI environments. It is backed by a parallel-native multi-device Docker infrastructure and efficiency-oriented metrics. We also introduced ParaGUI, a planner–worker agent that exploits the parallel tool-calling capability of modern LLMs. ParaGUI reaches up to $2.78\times$ parallelism on web search, with an overall task-weighted value of $2.13\times$ across ParaGUIBench. On the full 233-task benchmark, it raises success rate from 33.5% (the strongest serial baseline, Claude Sonnet 4.6) to 46.4% at roughly half the critical-path steps and less than half the tokens. When run in a serial (single-worker) configuration, it even outperforms a strong GUI-only baseline on the standard OSWorld benchmark (59.1% vs. 51.5%), with its only structural shortfall on the applications that demand the most visual continuity, Chrome and GIMP.

a) Limitations: Our analysis surfaces four limitations. First, *the worker backbone, not the planner, is the dominant bottleneck*. Most ParaGUI failures are confident-but-wrong completions whose leading cause is single-worker GUI execution rather than running out of steps; because the planner sees only an over-confident success signal (Table VIII), it cannot detect a dead-end trajectory and keeps dispatching. These undetected worker failures both cap capability and amplify the token overhead vs. the token-efficient Seed-1.8 baseline ($2.1\times$ on the full 233-task benchmark, though still $0.4\times$ vs. the token-heavier Claude Sonnet 4.6). Second, the planner delegates

disjoint-region assignment entirely to in-context reasoning, so current LMMs struggle to keep sub-tasks disjoint when they share a document, yielding decomposition and aggregation errors. Third, the synchronous round barrier makes ParaGUI sensitive to uneven sub-task lengths, because the planner cannot start the next round until every worker in the current round returns. Fourth, our experiments cover a limited set of LMM backbones: GPT-5.4 on the planner side; Seed-1.8, Qwen3-VL-235B-A22B-Instruct, Holo3-35B-A3B, Claude Sonnet 4.6, and Kimi K2.5 as workers or baselines. We leave broader coverage of worker and planner backbones to future work.

b) Future Work: These limitations point to several concrete directions. A natural next step is *adaptive task allocation*: a gating mechanism that switches between parallel and serial execution based on task complexity. This is most promising on information-retrieval tasks, which exhibit the largest parallelism headroom (web search reaches $P=2.78$; Table VI). *Worker-side calibration* training (e.g. reinforcement-learning (RL)-trained abstention) directly targets the over-confident commitment shown in Table VIII, a major driver of ParaGUI’s token overhead vs. Seed-1.8 GUI-only; planner-side RL over the round-based dispatch interface targets disjoint-region assignment and earlier termination of dead-end branches. Our OSWorld analysis points to a further direction: letting each worker persist its own visual frames—not merely the planner’s textual relay—across rounds, which is one promising way to recover the visual continuity that text-mediated dispatch loses on vision-heavy applications such as Chrome and GIMP. Action-conditioned world models studied in other visual decision domains [40] suggest one possible route to maintaining such visual state across dispatch rounds, although adapting them to GUI environments remains open. More broadly, allowing workers to exchange more than file artifacts—for example, through structured messaging or shared memory—would extend the parallel paradigm beyond storage-only synchronization. Asynchronous dispatch and adaptive load balancing are natural extensions that directly target the synchronous round barrier noted above.

REFERENCES

- [1] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, “OpenAI GPT-5 system card,” *arXiv preprint arXiv:2601.03267*, 2026.
- [2] R. Appel, P. McCrory, A. Tamkin, M. McCain, T. Neylon, and M. Stern, “Anthropic economic index report: Uneven geographic and enterprise ai adoption,” *arXiv preprint arXiv:2511.15080*, 2025.
- [3] J. M. S. Wong, “Speculative futures of education: utopian and dystopian scenarios envisioned by ChatGPT, Gemini, and DeepSeek,” *Discover Education*, vol. 4, no. 1, p. 261, 2025.
- [4] I. Ahmed, S. Islam, P. P. Datta, I. Kabir, N. U. R. Chowdhury, and A. Haque, “Qwen 2.5: A comprehensive review of the leading resource-efficient LLM with potential to surpass all competitors,” *Authorea Preprints*, 2025.
- [5] K. Team, T. Bai, Y. Bai, Y. Bao *et al.*, “Kimi k2.5: Visual agentic intelligence,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.02276>
- [6] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, “Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 9737–9757, Jun. 2025.
- [7] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey on vision-language-action models for embodied AI,” *IEEE Trans. Neural Netw. Learn. Syst.*, 2026, early access.

- [8] R. Abhyankar, Q. Qi, and Y. Zhang, “OSWorld-Human: Benchmarking the efficiency of computer-use agents,” *arXiv preprint arXiv:2506.16042*, 2025.
- [9] J. Cheng, H. Kang, Y. Shao, N. Li, P. Chen, R. Wang, S. Long, X. Yang, and L. Ma, “Survey on efficient large language models: Principles, algorithms, applications, and open issues,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 37, no. 5, pp. 2025–2045, May 2026.
- [10] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 52 040–52 094, 2024.
- [11] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [12] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried, “VisualWebArena: Evaluating multimodal agents on realistic visual web tasks,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 881–905.
- [13] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a generalist agent for the web,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 28 091–28 114, 2023.
- [14] R. Bonatti, D. Zhao, F. Bonacci, D. Dupont, S. Abdali, Y. Li, Y. Lu, J. Wagle, K. Koishida, A. Buckner, L. Jang, and Z. Hui, “Windows Agent Arena: Evaluating multi-modal OS agents at scale,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 4874–4910.
- [15] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. Bishop, W. Li, F. Campbell-Ajala *et al.*, “AndroidWorld: A dynamic benchmarking environment for autonomous agents,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [16] S. Kapoor, B. Stroebel, P. Kirgis, N. Nadgir, Z. S. Siegel, B. Wei, T. Xue, Z. Chen, F. Chen, S. Utpala *et al.*, “Holistic agent leaderboard: The missing infrastructure for AI agent evaluation,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [17] S. Kara, F. Faisal, and S. Nath, “WABER: Evaluating reliability and efficiency of web agents with existing benchmarks,” in *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025.
- [18] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, “An LLM compiler for parallel function calling,” in *Forty-first International Conference on Machine Learning*, 2024.
- [19] J. Yu, Y. Ding, and H. Sato, “DynTaskMAS: A dynamic task graph-driven framework for asynchronous and parallel LLM-based multi-agent systems,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 35, no. 1, 2025, pp. 288–296.
- [20] S. Zhang, X. Ma, Z. Cao, Z. Zhang, and H. Zhao, “Plan-over-Graph: Towards parallelizable LLM agent schedule,” *arXiv preprint arXiv:2502.14563*, 2025.
- [21] B. Niu, Y. Song, K. Lian, Y. Shen, Y. Yao, K. Zhang, and T. Liu, “Flow: Modularized agentic workflow automation,” in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [22] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [23] Q. Wang, T. Wang, Z. Tang, Q. Li, N. Chen, J. Liang, and B. He, “MegaAgent: A large-scale autonomous LLM-based multi-agent system without predefined SOPs,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- [24] X. Wu, Y. Zhu, C. Chen, and C. Chen, “GCM: Interpretable multiagent reinforcement learning via graph cooperation modeling,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 37, no. 4, pp. 1823–1837, Apr. 2026.
- [25] Y. Hu, Y. Yao, J. Chen, Z. Wang, Q. Jia, and Y. Pan, “Solving scalable multiagent routing problems with reinforcement learning,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 11, pp. 19 604–19 618, Nov. 2025.
- [26] C. Zhang, L. Li, H. Huang, C. Ni, B. Qiao, S. Qin, Y. Kang, M. Ma, Q. Lin, S. Rajmohan *et al.*, “UFO³: Weaving the digital agent galaxy,” *arXiv preprint arXiv:2511.11332*, 2025.
- [27] T. Qin, Q. Chen, S. Wang, H. Xing, K. Zhu, H. Zhu, D. Shi, X. Liu, G. Zhang, J. Liu *et al.*, “Flash-Searcher: Fast and effective web agents via DAG-based parallel execution,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [28] F. Lin, E. La Malfa, V. Hofmann, E. M. Yang, A. Cohn, and J. B. Pierrehumbert, “Graph-enhanced large language models in asynchronous plan reasoning,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [29] G. Gonzalez-Pumarega, L. S. Yean, N. Sunkara, and S. Choudhury, “Robotouille: An asynchronous planning benchmark for LLM agents,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [30] K. Shi, Z. Zhang, S. Huang, A. Nie, Z. Fang, Q. Wang, L. Chen, H. Chen, Z. Chen, and F. Zhao, “AsyncTool: Evaluating the asynchronous function calling capability under multi-task scenarios,” *arXiv preprint arXiv:2605.27995*, 2026.
- [31] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, D. Z. Wang, Z. Wang, C. Qian, R. Tang, H. Ji *et al.*, “MultiAgentBench: Evaluating the collaboration and competition of LLM agents,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 8580–8622.
- [32] H. Sun, S. Zhang, L. Niu, L. Ren, H. Xu, H. Fu, F. Zhao, C. Yuan, and X. Wang, “Collab-Overcooked: Benchmarking and evaluating large language models as collaborative agents,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 4922–4951.
- [33] S. Liu, M. Liu, H. Zhou, Z. Cui, Y. Zhou, Y. Zhou, J. Gao, H. Zhou, Y. Yang, W. Fan, P. Zhang, G. Zhang, J. Shi, W. Xuan, J. Huang, S. Luo, F. Wu, H. Qi, Q. Zeng, J. Wang, A. Feng, J. Lv, S. Jiang, Z. Ren, W. Zhou, Z. Yin, W. Zhang, G. Li, W. Yu, L. Ma, L. Bai, Q. Lin, M. Song, and D. Tao, “Veriweb: Verifiable long-chain web benchmark for agentic information-seeking,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.04026>
- [34] R. Peeters, A. Steiner, L. Schwarz, J. Y. Caspary, and C. Bizer, “WebMall – a multi-shop benchmark for evaluating web agents,” in *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2026.
- [35] OpenAI, “Introducing GPT-5.4,” <https://openai.com/index/introducing-gpt-5-4/>, 2026, accessed: 2026-03-05.
- [36] ByteDance, “Seed-1.8 Model Card: Technical Specifications and Capabilities,” ByteDance, Tech. Rep., 2026, technical Report. [Online]. Available: <https://lfl3-static.bytednsdoc.com/obj/eden-cn/lapzild-tss/ljhwZthlaukjlkulzlp/research/Seed-1.8-Modelcard.pdf>
- [37] Qwen Team, “Qwen3-vl technical report,” *arXiv preprint arXiv:2511.21631*, 2025.
- [38] H Company, “Holo3 – open foundation models for navigation and computer use agents,” <https://huggingface.co/Hcompany/Holo3-35B-A3B>, 2026, accessed: 2026-06-04.
- [39] Anthropic, “Claude Sonnet 4.6,” <https://www.anthropic.com/claude/sonnet>, 2026, accessed: 2026-06-04.
- [40] P. Ren, K. Zhang, H. Zheng, Z. Li, Y. Wen, F. Zhu, S. Ma, and X. Liang, “Surfer: A world model-based framework for vision-language robot manipulation,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 12, pp. 20 092–20 104, Dec. 2025.

APPENDIX A

EXTENDED EXPERIMENTAL ANALYSIS

A. Efficiency by Parallel-Execution Pattern

A breakdown by parallel class $\times$ category (Table XI) shows where the largest parallelism gains accrue. The biggest gains come from web search and online shopping within *parallel_independent* and *parallel_dependent*: in these categories, each sub-task admits a separate browser context. The search & write category is limited by a shared-document bottleneck: its sub-tasks edit the same file, so parallelism reaches only 1.47 and SR drops to 10%. Concurrent edits on one document expose a limitation of in-context planning—it cannot reliably keep workers’ edits within disjoint regions.

B. Extended Ablation Discussion

Table XII reports the full per-category breakdown of the ablation summarized in Table IX, and Figure 5 visualizes the same comparison in the capability–efficiency plane.

The 51-task subset approximates the full benchmark’s domain mix (IR 33.3%, Shop 39.2%, FileOps 17.6%, WebNav

TABLE XI
PARAGUI ON PARAGUIBENCH BROKEN DOWN BY PARALLEL CLASS × CATEGORY (CELLS WITH $n \leq 2$ ARE REPORTED BUT SHOULD BE INTERPRETED WITH CAUTION). THE PARALLEL CLASS IS A TASK’S ANNOTATED INTRINSIC DEPENDENCY STRUCTURE; THE ACTUALLY DISPATCHED PARALLELISM DEGREE CAN STILL EXCEED 1 WHEN THE PLANNER OVER-DISPATCHES, SO A FEW *serial*-CLASS CELLS (E.G. WEB NAVIGATE, $n=2$) REPORT $P > 1$.

Parallel-pattern class	Category	#	SR (%)	Critical path (avg)	Para.	Tokens (M)
parallel_independent	Web Search	30	36.67	31.60	2.62	0.83
	File Search	8	87.50	20.25	1.20	0.21
	Online Shopping	75	52.00	28.49	2.51	0.69
	File Operation	30	33.33	47.27	1.98	0.96
	Web Navigate	11	63.64	28.55	1.59	0.43
	Search & Write	10	10.00	59.20	1.47	0.87
parallel_dependent	Web Search	32	46.88	47.44	2.95	1.38
	Online Shopping	8	75.00	57.50	1.64	0.93
	File Operation	10	30.00	71.20	1.26	0.96
serial	Web Search	3	33.33	20.67	1.00	0.19
	File Search	4	50.00	20.75	1.23	0.26
	Online Shopping	8	62.50	47.50	1.00	0.49
	File Operation	2	50.00	80.50	1.00	0.79
	Web Navigate	2	0.00	33.00	1.71	0.55

3.9%, SearchWrite 5.9%). Relative to the Seed-1.8 serial baseline (17.6%, 28.3 steps, 0.29M tokens), ParaGUI (39.2%, 33.3 steps, 0.73M tokens) lifts SR by +21.6 points while spending a few extra steps and roughly $2.5\times$ more tokens. The step parity, rather than a step reduction, traces to a baseline behavior pattern. Seed-1.8 frequently commits to a confident-but-wrong answer well before the step budget: averaged over failed tasks, Seed-1.8 spends 36.6 steps versus 80.1 for the well-calibrated Claude Sonnet 4.6 (Table VIII), with the search & write row at 40.9 versus 47.1. Almost none of these early terminations are voluntary `fail()` calls; they are confident `done(answer)` that the evaluator rejects. This pattern suppresses Seed-1.8’s step and token counts without improving SR. ParaGUI continues dispatching through additional rounds, lifting failed-task step counts to 42.8 on average—closer to well-calibrated behavior despite using a Seed-1.8 worker (Table VIII); these extra rounds give the planner more chances to correct a worker’s output, though they do not verify the final environment state. The clean step reduction visible on the full benchmark (38.7 vs. 75.9) appears precisely once the comparison baseline switches to Claude Sonnet 4.6, which does *not* exhibit early confident-but-wrong commitment. Isolating SCD directly, the $-$ SCD variant reaches only 27.5% versus 39.2% for the full system; this confirms SCD as the main capability lever.

Holding the worker fixed to Seed-1.8, we vary the planner. A Seed-1.8 planner (plan-seed-1.8) matches the GPT-5.4 planner in overall success (39.2%). A Kimi K2.5 planner (plan-kimi) trails only modestly (33.3%). Both remain far above the 17.6% serial baseline. The planner model is therefore *not* the capability bottleneck on this subset. Once decomposition and SCD are in place, even a Seed-1.8 planner produces sub-tasks the worker can execute. The GPT-5.4 planner helps efficiency: it reaches 39.2% in 33.3 critical-path steps, against 40.8 (plan-seed-1.8) and 45.6 (plan-kimi). These are tighter, less redundant decompositions.

We next fix the planner to GPT-5.4 and vary the worker. A Claude Sonnet 4.6 worker (claude-gui) attains the highest SR (47.1%), followed by Seed-1.8 (39.2%) and Kimi K2.5 (kimi-gui, 35.3%). All three more than double the 17.6% serial

baseline. Seed-1.8 is the most efficient (33.3 steps, 0.73M tokens, versus 54.0/1.37M for Kimi and 50.0/1.98M for Claude). It offers the best accuracy–efficiency trade-off. Claude is the strongest worker when capability matters more than cost.

Varying the parallelism degree shifts *which* tasks are solved without monotonically improving aggregate capability. Across $N=1/3/5$, the three configurations all solve the same 18 tasks; $N=1$ uniquely solves 3 more tasks that $N=3/5$ miss, while $N=3$ and $N=5$ each unlock 2 tasks that $N=1$ misses. Two opposing mechanisms are visible. (i) Extra parallel branches incur additional dispatch errors that cost $N=3/5$ the three $N=1$ -only tasks: the planner has to split a coherent trajectory across workers, and any single dispatch boundary error fails the whole task. (ii) The same parallelism unlocks decompositions that a single worker cannot complete within the 25-step per-invocation cap: an under-budgeted serial trajectory simply runs out of room. The two effects nearly cancel on this subset, leaving aggregate SR essentially flat (41.2 / 39.2 / 39.2). What N changes monotonically is *efficiency*: the critical-path step count falls $60.5 \rightarrow 39.0 \rightarrow 33.3$. The planner–worker pair sets the capability ceiling; larger N gives up little accuracy for a substantially shorter critical path.

C. Visual-History Budget: Why ParaGUI Saves Tokens vs. Claude Sonnet 4.6

A natural concern is whether ParaGUI’s token advantage over Claude Sonnet 4.6 (Table V) is an artifact of the visual-history configuration: ParaGUI’s Seed-1.8 worker retains 3 screenshots, while Claude’s default policy retains up to 10. We test this directly by restricting Claude Sonnet 4.6 to a 3-screenshot budget (matching Seed-1.8’s policy) on the 51-task subset; the result is the row labelled “GUI-only (image=3)” in Table XII. The restriction does *not* reduce Claude’s token cost. Overall SR drops slightly (35.3% $\rightarrow$ 33.3%), critical-path steps *increase* (67.1 $\rightarrow$ 78.0), and total tokens *increase* (1.64M $\rightarrow$ 1.76M): with fewer screenshots in context the agent compensates by re-grounding for more steps, and the extra steps more than offset the per-step token savings. Under this normalized visual-history budget, ParaGUI’s token use relative to Claude drops from $0.45\times$ to $0.42\times$ on this 51-task subset (0.73M/1.64M vs. 0.73M/1.76M; Table XII), widening its token advantage. This rules out a pure visual-context-budget account of the token gap and indicates that Claude’s default 10-screenshot retention is closer to its own best operating point on these tasks. The token gap reported in the main results therefore reflects intrinsic per-call cost and trajectory length, not under-allocated visual context.

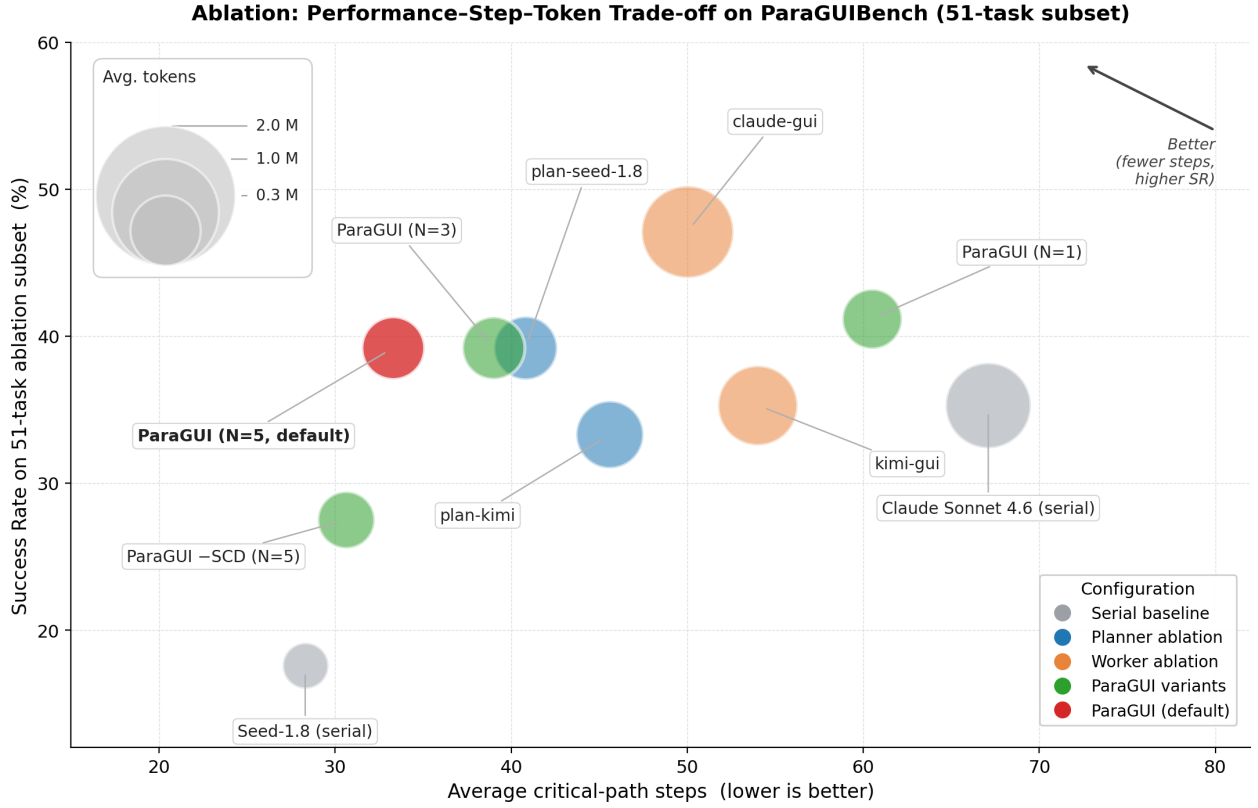


Fig. 5. Ablation on the 51-task subset in the capability–efficiency plane. Vertical: Success Rate ($\uparrow$); horizontal: avg. critical-path steps ($\downarrow$); bubble area $\propto$ avg. per-task tokens. Colour groups the comparison type (serial baseline / planner / worker / –SCD / N -sweep); the dotted path traces $N=1 \rightarrow 3 \rightarrow 5$, where steps shrink at near-constant SR. ParaGUI ($N=5$, default) is the fastest high-SR Seed-1.8-worker variant, while $N=1$ obtains slightly higher SR at substantially more steps. Numbers in Table XII.

TABLE XII

FULL PER-CATEGORY ABLATION ON THE 51-TASK SUBSET, WITH INFORMATION RETRIEVAL (IR) SPLIT INTO **FILE SEARCH** AND **WEB SEARCH**. SR, STEPS, TOKENS FOLLOW TABLE IX. “—”: NO PLANNER (SERIAL). PARA GUI ($N=5$): DEFAULT SYSTEM; –SCD: NAIVE DISPATCH, ISOLATING SCD’S CONTRIBUTION. WEB SEARCH INCLUDES THE SINGLE VISUALSEARCH TASK, FOLLOWING THE MAIN-TABLE TAXONOMY.

Success Rate											
Configuration	planner	worker	File Search (7)	Web Search (10)	Shop (20)	WebNav (2)	FileOps (9)	SearchWrite (3)	All (51)	steps	Tokens (M)
Serial baseline											
GUI-only	—	Seed-1.8	0.0%	30.0%	20.0%	100.0%	0.0%	0.0%	17.6%	28.3	0.29
GUI-only	—	Claude Sonnet 4.6	28.6%	30.0%	40.0%	100.0%	33.3%	0.0%	35.3%	67.1	1.64
GUI-only (image=3)	—	Claude Sonnet 4.6	57.1%	10.0%	50.0%	100.0%	0.0%	0.0%	33.3%	78.0	1.76
Parallel variants (ablating planner or worker)											
plan-seed-1.8	Seed-1.8	Seed-1.8	71.4%	30.0%	35.0%	50.0%	44.4%	0.0%	39.2%	40.8	0.74
plan-kimi	Kimi K2.5	Seed-1.8	57.1%	40.0%	30.0%	50.0%	22.2%	0.0%	33.3%	45.6	0.88
kimi-gui	GPT-5.4	Kimi K2.5	71.4%	30.0%	30.0%	100.0%	22.2%	0.0%	35.3%	54.0	1.37
claude-gui	GPT-5.4	Claude Sonnet 4.6	71.4%	50.0%	50.0%	100.0%	22.2%	0.0%	47.1%	50.0	1.98
ParaGUI —SCD (N=5)	GPT-5.4	Seed-1.8	71.4%	20.0%	25.0%	100.0%	0.0%	0.0%	27.5%	30.6	0.55
ParaGUI (N=1)	GPT-5.4	Seed-1.8	85.7%	30.0%	40.0%	100.0%	22.2%	0.0%	41.2%	60.5	0.63
ParaGUI (N=3)	GPT-5.4	Seed-1.8	71.4%	30.0%	35.0%	100.0%	33.3%	0.0%	39.2%	39.0	0.71
ParaGUI (N=5, default)	GPT-5.4	Seed-1.8	71.4%	30.0%	40.0%	100.0%	22.2%	0.0%	39.2%	33.3	0.73